République Algérienne Démocratique et populaire Ministère de l'enseignement Supérieure
et de la echerche Scientifique
Université M'Hamed Bougara - Boumerdés

Faculté des Siences Département d'informatique

Niveau : Master informatique (I2A)

Module : Web Acance et micro-Services

Rapport de Project

Group : 2

Réalisé par:

TARIKET Mohamed 222231388506
LOUAHCHI Mohamed 222231354603
AZZOUK Abdelmalek 22223163360
ACHELI Yassine 222231633308

Année universitaire : 2025/2026

1. Project Overview and Architecture

Project Overview

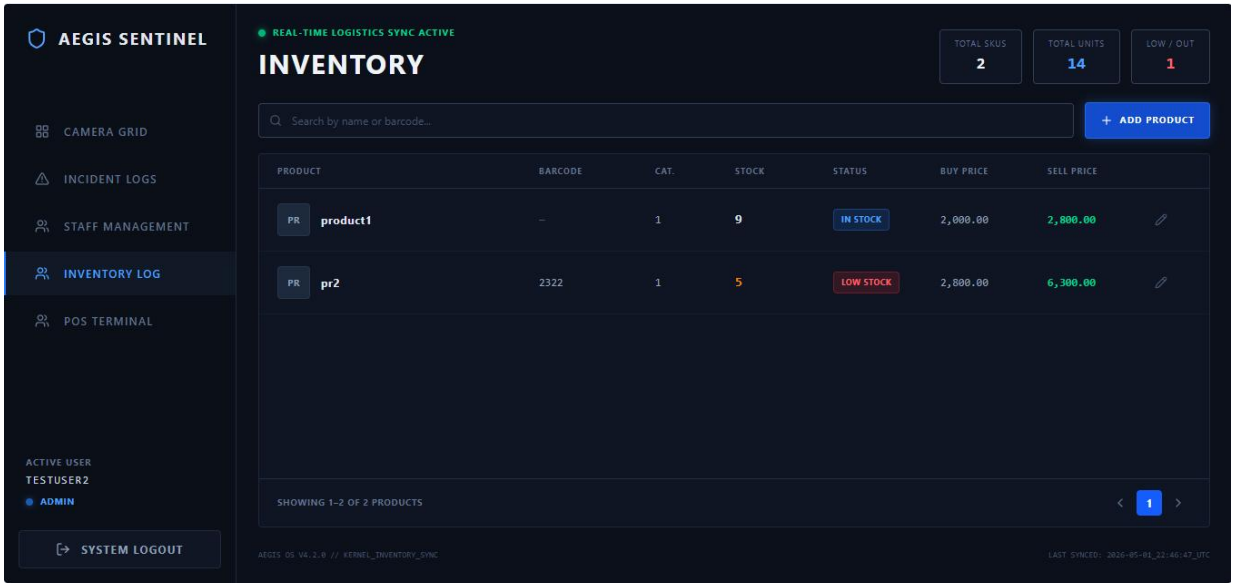
We build a backend system based on microservices. The system manages retail operations and security monitoring. Each service focuses on one task. This improves scalability and control.

Main Features
We managed core business data:

- Product management
- Sales management
- Employee management
- Camera management

We added an automated security layer:

- Deep learning model detects shoplifting events
- System records visual evidence for each detection



System Architecture
We split the system into independent services. Each service runs alone and handles one domain.

Microservices Breakdown
Deploy six services:

- `user_service`
- `product_service`
- `sales_service`
- `employee_service`
- `auth_service`
- `model_worker_service`

Each service:

- Runs independently
- Uses its own database
- Communicates through HTTP requests

2. Request Lifecycle

We processed each secured request in steps:

- Receive request
- Validate token
- Check role
- Reach endpoint acces
- Verify permissions
- Execute database operation using ORM
- Return response

3. Authentication

Security and Authentication
We centralize security in `auth_service`.

Core Mechanisms

- JWT for stateless authentication
- Role based access authorization

Authentication Workflow

- User log in and receives access and refresh tokens
- The frontend securely stores authentication tokens in local storage
- Each request includes Authorization header with access token

```
Authorization: Bearer <access_token>
```

4. Service Registry and Discovery

We use Consul as a service registry.At startup, each service registers its address in Consul. Other services retrieve this information from Consul to locate and communicate with the required service.

This setup eliminates hardcoded IP addresses and improves flexibility, especially when scaling or updating services.

.

5. Media Handling

The security_service is responsible for managing shoplifting evidence media. The workflow is designed to ensure reliable capture, storage, and access to recorded footage.

- The detection model automatically records a video clip for each detected incident.
- The backend server stores the generated clips locally in a structured media directory.
- The application configures MEDIA_URL to define a public endpoint for accessing stored media files.
- Each clip is uniquely identified and linked to its corresponding detection event for traceability.

6. CORS Configuration

Allow communication between frontend and services:

- Enable CORS
- Accept requests from different domains and ports
- Add the custom header ngrok-skip-browser-warning to the allowed CORS headers in the Django configuration.

```
ALLOWED_HOSTS = ["*"]
CORS_ALLOW_ALL_ORIGINS = True
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOW_HEADERS = list(default_headers) + [
    "ngrok-skip-browser-warning",
]
```

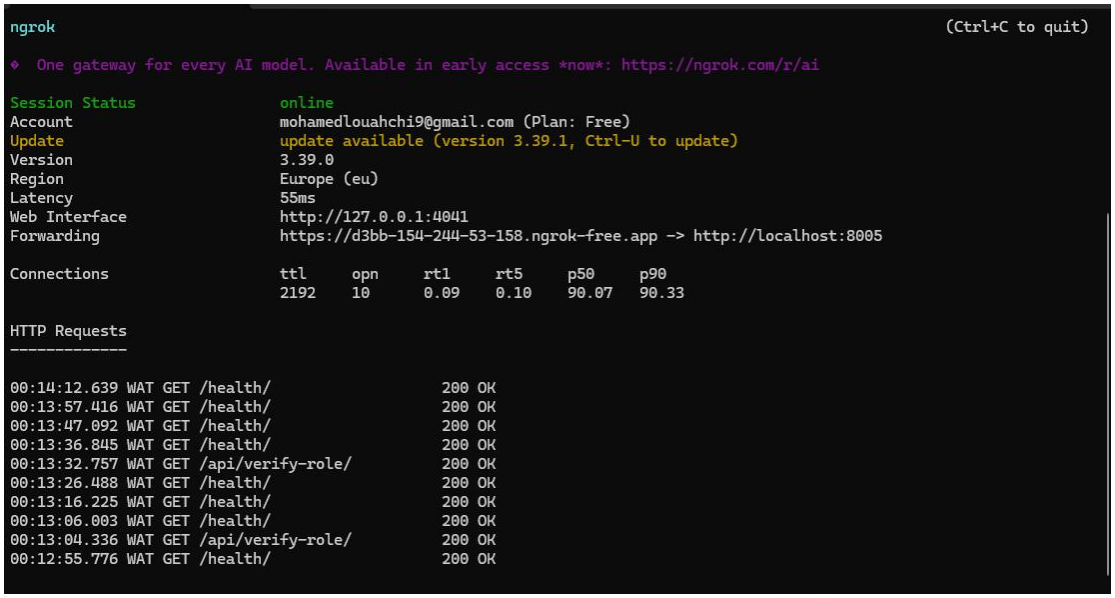
7. Deploiement multi-serveurs

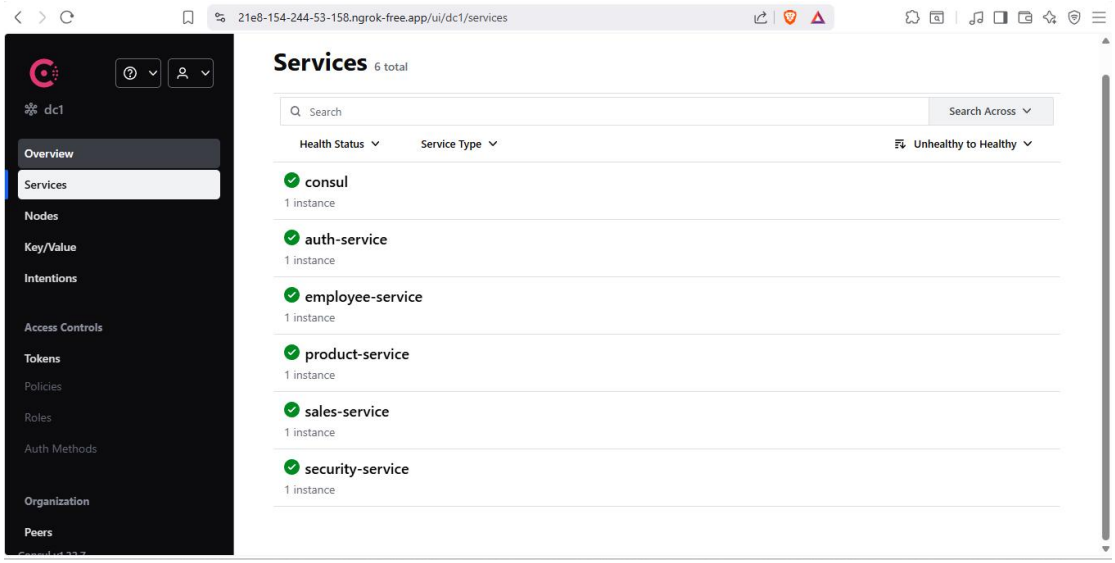
We prepared each server with the same steps:

- Install service dependencies
- Run database migrations
- Start the service on a defined port

We Exposed each service for external access:

- Run Ngrok on the same port
- Retrieve the public URL generated by ngrok and configure it as the host URL for the service
- Next, the service is registered with Consul, enabling it to be discovered and accessed by other services through the service registry





Running the frontend

Start the system in sequence:

- Instal depenensis

  `npm install`

- Start the frontend with

  `npm run dev`